ELARA: Energy-Efficient and Latency-Aware Service Routing with Adaptive Replica Redeployment for Microservices in Space Computing Power Networks

Anonymous Authors

Abstract—The rapid advancement of the Space Computing Power Network (SCPN), comprising numerous low Earth orbit (LEO) satellites, has enabled the execution of complex in-orbit observation and computation tasks. To facilitate this, microservice architecture decomposes large, monolithic remote sensing and image processing applications into independent, schedulable, and reusable services deployed in a distributed manner across LEO satellites. However, integrating these capabilities presents significant challenges arising from the limited computing resources, constrained power provisions of individual satellites, and the dynamic nature of space network topology and channel conditions. This paper addresses these challenges by proposing an energy-aware service routing algorithm for microservices within the SCPN. The algorithm aims to minimize the energy consumed by the procedures of processing and transmitting service data in both computation and communication. By modeling the service routing problem as a Directed Steiner Tree (DST) and implementing heuristic-based solutions, our algorithm effectively responds to the diverse service patterns requested by terrestrial clients. Comprehensive experimental results demonstrate that our approach significantly outperforms existing methods in terms of energy efficiency, robustness, stability, and adaptability.

Index Terms—Energy, Microservice, Routing, Space Network

I. Introduction

In recent years, large-scale LEO satellite networks have garnered widespread attention as one of the recognized highlights of 6G technology [1]. Modern LEO mega-constellations, consisting of hundreds to thousands of LEO satellites connected by inter-satellite links (ISLs), enable the globally ubiquitous connections [2]. Meanwhile, advancements in remote sensing and chip technology have enabled modern LEO satellites to be equipped with high-performance imaging and intelligent computing payloads [3], [4], thus supporting ubiquitous intelligent services such as weather forecasting, environmental monitoring, and national security [5], [6]. Hundreds to thousands of satellite computing nodes collectively form the SCPN, where the strategic allocation of sensing, communication, and computing resources to efficiently fulfill diverse intelligent service requests has emerged as a critical challenge in the context of 6G integrated space-air-ground networks [7].

Particularly, due to the large-scale deployment of LEO satellites, advancements in satellite remote sensing and data analytics technology, and the high image resolution afforded by the lower orbital altitude, LEO satellite remote sensing services have demonstrated unprecedented momentum. These services continuously generate massive amounts of data featuring multiple formats and modalities, catering to extensive user demands [8]–[10]. This paper illustrates these application scenarios as depicted in Fig. 1.

Due to the enormous and continuously growing volume of remote sensing data, downloading all satellite data to the ground for centralized processing consumes substantial satellite-to-ground communication resources and may cause congestion for other services. Therefore, leveraging onboard computational resources to perform partial or complete processing of remote sensing tasks and then downloading the computational results to the ground can save communication resources and mitigate privacy and security risks associated with downloading raw data [11]. Furthermore, considering the potential computational resource constraints of a single satellite, the multi-satellite collaborative computing paradigm has emerged as a mainstream research focus [12], [13]. Specifically, thanks to the development of containerization technologies and the evolution of microservice architectures [14]–[16], the functional modules of the user applications, which are traditionally deployed in terrestrial data centers to process raw data postback from satellites, [17], can now be decomposed into stateless microservices and packaged as independent and isolated containers [18]. These services can then be dispatched to computing nodes of the SCPN for on-board computation so as to reduce the transmission delay caused by limited bandwidth and vulnerable channel conditions between ground and space networks [19], [20].

However, unlike terrestrial computing nodes with adequate computing resources and power supply. The SCPN nodes present unique limitations, i.e., constrained computing resources and poor power supply. On the one hand, satellites can only be equipped with limited computing and storage resources as a consequence of the constraints of size and weight, as well as the damage risk caused to circuit boards by intense radiation in space [21]. On the other hand, the energy supply for satellites is primarily derived

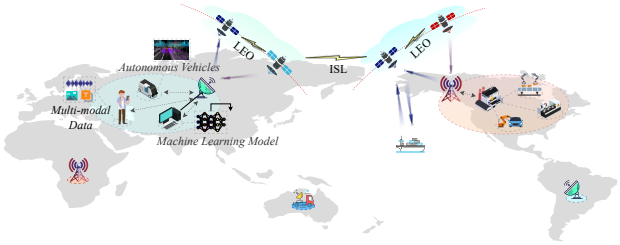


Fig. 1: Space and terrestrial network topology.

from solar energy harvesting, which is utilized for various satellite functions, including attitude control, transceivers, cameras, and processors. Furthermore, considering the area of solar panels, the efficiency of the materials used for energy harvesting, and hardware degradation, the battery power available for satellites is extremely limited compared to the energy supply available to terrestrial servers. [22], [23].

Hence, different from terrestrial settings [24]–[26], the energy efficiency should be given high priority and regarded as the main concern when devising the routing schemes in the SCPN [22], [27], [28]. To this end, this work presents an energy-aware service routing algorithm for microservices in the SCPN, aiming to minimize energy consumption in both computation and communication to meet the task requirements of terrestrial clients. Specifically, we first model the service routing problem as a Directed Steiner Tree (DST) on the directed graph, which is a classic combinatorial optimization problem and has been proved to be NP-hard [29], [30], then design a series of heuristic-based algorithms to obtain the approximate optimal solution for the service routing problem in the SCPN.

Our main contributions are summarized as follows:

- Service Routing Modeling in the SCPN: We identify satellite energy consumption as a critical bottleneck of microservice routing in the SCPN. To this end, we meticulously model the satellite energy consumption associated with microservice communication and computation as well as mathematically formulate the energy resource optimization problem for satellite microservices.
- Augmented Satellite-Microservice Subgraph Construction: We propose an innovative augmented satellite-microservice subgraph construction method, which equivalently transforms the original problem into a classical DST problem in graph theory, thereby elucidating the mathematical essence of this engineering challenge. In addition to microservice scenarios, this method is also applicable to other routing problems that jointly optimize the computational and communication resource costs.
- Heuristic Algorithms for the DST Problem: We implement a series of heuristic algorithms to approximate the optimal solution for the NP-hard DST problem. Extensive experiments demonstrate that our proposed algorithm not only significantly outperforms baseline algorithms in energy efficiency but also exhibits remarkable stability, thereby validating the efficacy of our augmented graph construction methodology.

The remainder of this paper is organized as follows. Section II reviews the related works. Section III presents the system models. The problem formulation is introduced in Section V. Section VI details the proposed heuristic algorithms. Section VII shows and analyses the experiment results. Section VIII draws the conclusion.

II. Related Work

Recent advancements in satellite technology have significantly promoted its computing and transmitting capabilities, enabling satellites to undertake some elementary tasks [8]. The construction of ISLs, in particular, enables interconnected satellites to form the SCPN with pooled computing resources, supporting intricate missions with enhanced performance demands [13], [21], [31] and facilitating a global response to requests from anywhere at any time [32], [33]. In addition, careful energy management is necessary for satellites to realize sustainable development [22], [23].

Considering the discrete distributed computing nodes (as satellites) with limited computing power in the SCPN, researchers have migrated the monolithic applications (such as 3D reconstruction and inference model), which are traditionally deployed on ground data centers, to space networks by decomposing them into microservices, which are characterized by being independent, scalable, reusable, and stateless [16], [18], [34]. These microservices can be scaled and deployed on the satellites through platform-independent containerization technology, which encapsulates these isolated microservices into containers that are transparent to the installed computing platform on satellites [14], [35].

Extensive research has been conducted on service routing algorithms, aiming at the problems of node selection and routine planning. Fu et al. [28] proposed a dynamic programming approach to drive the optimal policy on energy allocation for a single satellite in response to user requests. It fails to consider the multi-satellite cases. The work [36] designed a joint edge server placement and service deployment model for SAGIN, aiming to minimize the average response latency and energy cost. The work [22] developed a series of heuristic algorithms to prolong the serving life of satellites by switching part of routers installed on satellites to sleep mode, and resolving the energy-efficient routing policies. Efficient but shortsighted, these proposed algorithms cannot adapt to diverse routing needs, for example, the treed topology raised by requested missions, which demands integrated computing and communication routing as well as multi-satellite collaboration.

III. System Model

We consider a space computing power network (SCPN) composed of a Walker-Delta LEO constellation, onboard computing resources, and laser inter-satellite links (ISLs). User tasks are represented as ordered microservice chains. Each microservice can have multiple stateless replicas deployed on different satellites, and a request may therefore be executed collaboratively by multiple satellites.

A. LEO Constellation and Time-slotted Topology

A typical Walker-Delta constellation is parameterized by $\mathcal{W} = (A/P/G, H, I)$, where A is the total number of satellites, P is the number of orbital planes, G is the inter-plane phasing factor, H is the orbital altitude, and I is the inclination. Consequently, each orbital plane accommodates $N = A/P$ satellites. A satellite node is uniquely identified by its orbital plane index p and its intra-plane position index n, denoted by $v_{p,n}$. The set of satellite nodes in the constellation is thus formally defined as $\mathcal{V} = \{v_{p,n} \mid p = 1, \ldots, P,\ n = 1, \ldots, N\}$. To capture the high dynamism of the satellite network, we discretize the constellation's operational period into a set of time slots, denoted by $\mathcal{T} = \{0, 1, \ldots, T-1\}$, with a uniform slot duration Δ. For any continuous-time instant τ, the corresponding time slot index is mapped as $s(\tau) = \lfloor \tau/\Delta \rfloor \bmod T$. Adopting a snapshot-based modeling approach, we assume that the network state, including satellite positions, ISL availability, and nominal link capacities, remains quasi-static within a single time slot, and only evolves at the slot boundaries governed by orbital mechanics. Thus, the time-varying network topology during slot $t \in \mathcal{T}$ is formally defined as a dynamic graph $\mathcal{G}(t) = \big(\mathcal{V}, \mathcal{E}(t)\big)$, where $\mathcal{E}(t)$ represents the set of feasible ISLs at slot t.

B. ISL Communication Model

We assume that each satellite is equipped with four laser optical terminals: two positioned along the roll axis for intra-plane ISLs, and two along the pitch axis for cross-plane ISLs.

a) ISL Connectivity Constraints: Each satellite $v_{p,n}$ maintains two intra-plane ISLs with its adjacent neighbors v_{p,n^-} and v_{p,n^+}, where $n^{\pm} = ((n \pm 1 - 1) \bmod N) + 1$. Because satellites in the same plane share identical angular velocities, their relative distances remain time-invariant, yielding highly stable links.

Conversely, cross-plane ISLs are inherently dynamic. Constrained by the physical and kinematic limits of onboard laser communication terminals (LCTs), the existence of a cross-plane link $e = (u, v)$ at slot t is jointly dictated by line-of-sight (LoS) occlusion, mechanical tracking thresholds, and polar exclusions, formulated as:

$$e \in \mathcal{E}(t) \iff \begin{cases} d_{u,v}(t) \le d_{\max}, \\ \theta_{u,v}(t) \le \theta_{\max}, \\ \max\big(|\varphi_u(t)|, |\varphi_v(t)|\big) \le \varphi_{\max}. \end{cases} \tag{1}$$

In (32), the LoS distance $d_{u,v}(t)$ and relative tracking angle $\theta_{u,v}(t)$ are bounded by the terrestrial occlusion limit $d_{\max}$ and the hardware threshold $\theta_{\max}$, respectively. Furthermore, the third constraint enforces a polar exclusion zone at latitude boundary $\varphi_{\max}$ (e.g., 70°). This mechanism proactively tears down cross-plane links at high latitudes to bypass the severe pointing dynamics and extreme Doppler shifts inherent to polar regions.

b) ISL Capacity Model: Unlike ground networks with stable capacities, the transmission rate of laser ISLs is distance-dependent due to geometric propagation loss. Based on the Free-Space Optical (FSO) communication model, the received optical power $P_e^{\mathrm{rx}}(t)$ at satellite v transmitted from satellite u is formalized as:

$$P_e^{\mathrm{rx}}(t) = P^{\mathrm{tx}} G^{\mathrm{tx}} G^{\mathrm{rx}} \eta_{\mathrm{tx}} \eta_{\mathrm{rx}} \left(\frac{\lambda}{4\pi d_{u,v}(t)} \right)^2, \tag{2}$$

where P^{tx} is the nominal laser transmit power, G^{tx} and G^{rx} represent the optical gains of the transmitter and receiver apertures respectively, λ is the laser wavelength, and $\eta_{\mathrm{tx}}, \eta_{\mathrm{rx}}$ denote the optical efficiency factors. The real-time Signal-to-Noise Ratio (SNR) of the link e at time slot t is given as:

$$\gamma_e(t) = \frac{\left(R_{\mathrm{pd}} \cdot P_e^{\mathrm{rx}}(t)\right)^2}{\sigma_{\mathrm{sh}}^2 + \sigma_{\mathrm{th}}^2}, \tag{3}$$

where R_{pd} is the photodiode responsivity, while σ_{sh}^2 and σ_{th}^2 denote the shot and thermal noise variances, respectively. According to the Shannon-Hartley theorem, the achievable transmission capacity $R_e^0(t)$ is constrained by:

$$R_e^0(t) = \mathrm{B}_e \log_2\left(1 + \gamma_e(t)\right), \tag{4}$$

where B_e is the modulation bandwidth.

c) ISL Delay Model: The nominal rate $R_e^0(t)$ in (36) could be degraded in practice by transient channel fading and traffic contention. To capture these factors, we introduce a scaling factor $\eta_e(t) \in (0, 1]$ to formulate the effective data rate as $R_e(t) = \eta_e(t) R_e^0(t)$. When transmitting a data block of size B over link $e = (u, v)$ during slot t, the aggregate link-level communication delay comprises queuing, transmission, and propagation delays, formulated as:

$$T_e^{\mathrm{cm}}(B, t) = \tau_e^{\mathrm{qe}}(t) + \tau_e^{\mathrm{tr}}(t) + \tau_e^{\mathrm{pr}}(t). \tag{5}$$

Here, $\tau_e^{\mathrm{qe}}(t)$ denotes the queuing delay, which can be dynamically estimated via lightweight queue-length probing. The transmission delay is given by $\tau_e^{\mathrm{tr}}(t) = B/R_e(t)$, and the propagation delay is $\tau_e^{\mathrm{pr}}(t) = d_e(t)/c$, with c being the speed of light in vacuum.

C. Onboard Computing Model

Let F_v^0 denote the nominal computing capacity of satellite v. In practice, the available computational resources are often constrained by background payload tasks, operating system overhead, and thermal throttling.

To capture these hardware and software limitations, we introduce a computational efficiency factor $\xi_v(t) \in (0, 1]$ and define the effective dynamic computing capacity as $F_v(t) = \xi_v(t)F_v^0$.

Let $\tau_v^{\mathrm{qe}}(t)$ denote the queuing delay experienced before microservice m is scheduled for execution on satellite v. We formulate the total computation delay as:

$$T_m^{\mathrm{cp}}(v,t) = \tau_v^{\mathrm{qe}}(t) + \tau_v^{\mathrm{cp}}(t). \tag{6}$$

Here, the queuing delay $\tau_v^{\mathrm{qe}}(t)$ is a monotonically increasing function of the pending task queue length on satellite v at slot t. The actual execution delay is given by $\tau_v^{\mathrm{cp}}(t) = C_m/F_v(t)$, where C_m denotes the number of CPU cycles required by microservice m. Correspondingly, the computational energy consumption is defined as:

$$E_m^{\mathrm{cp}}(v,t) = P_v^{\mathrm{cp}} \frac{C_m}{F_v(t)}, \tag{7}$$

where P_v^{cp} is the operational computing power of the satellite.

D. LEO Microservice and Service Request Models

To facilitate flexible application management within the SCPN, complex applications are decoupled into a set of fine-grained, loosely coupled microservices, denoted by $\mathcal{M}$. Each microservice $m \in \mathcal{M}$ is characterized by its computational demand C_m (in CPU cycles), container image size I_m, and storage requirement S_m. For robust load balancing and fault tolerance, multiple replicas of each microservice are distributed across the satellite constellation.

We define a binary decision variable $x_{m,v}(t)$ to indicate if microservice m is deployed on satellite v during time slot t:

$$x_{m,v}(t) = \begin{cases} 1, & \text{if is deployed,} \\ 0, & \text{otherwise.} \end{cases} \tag{8}$$

Accordingly, the subset of satellite nodes hosting a valid replica of microservice m at slot t is given by:

$$\mathcal{R}_m(t) = \{v \in \mathcal{V} \mid x_{m,v}(t) = 1\}. \tag{9}$$

To ensure service availability while preventing excessive resource consumption, the total number of deployed replicas for any microservice should be bounded within predefined thresholds, $r_m^{\min}$ and $r_m^{\max}$:

$$r_m^{\min} \le \sum_{v \in \mathcal{V}} x_{m,v}(t) \le r_m^{\max}, \quad \forall m \in \mathcal{M}. \tag{10}$$

Furthermore, the aggregate storage footprint of all replicas hosted on a single satellite must not exceed its onboard storage capacity $S_v^{\max}$:

$$\sum_{m \in \mathcal{M}} x_{m,v}(t) S_m \le S_v^{\max}, \quad \forall v \in \mathcal{V}. \tag{11}$$

It is worth noting that all microservices operate in a stateless manner, which can be dynamically added, removed, or migrated without incurring the prohibitive communication overhead of inter-replica state synchronization.

Let $r = \langle v_s, m_1, m_2, \ldots, m_L, v_d, \tau_{\mathrm{in}} \rangle$ denote an incoming request with a deadline D_r. Here, v_s and v_d are the source and destination satellites, m_i is the i-th microservice in the sequential execution chain, and τ_{in} is the continuous arrival time. Let $B_{r,i}$ be the data volume that should be transmitted before invoking m_i, while $B_{r,L+1}$ denotes the final output routed back to v_d. Since a request can arrive at any arbitrary instant and its data transmission may span multiple topology slots, ELARA formulates this sequential service execution as a continuous-time online decision process. The detailed cross-slot execution dynamics, decision variables, and objective functions are formally presented in Section IV.

IV. Problem Formulation

A. Cross-slot Execution and Decision Variables

ELARA tracks each request in continuous time. Before executing microservice m_i, the state of request r is

$$\sigma_{r,i} = (u_{r,i}, \tau_{r,i}, i), \tag{12}$$

where $u_{r,i}$ is the satellite currently holding the input data for m_i and $\tau_{r,i}$ is the corresponding decision epoch. Initially, $u_{r,1} = v_s$ and $\tau_{r,1} = \tau_{\mathrm{in}}$.

For request r, stage i, and satellite v, the replica-selection variable is

$$z_{r,i,v} = \begin{cases} 1, & \text{if } m_i \text{ of } r \text{ is executed on } v, \\ 0, & \text{otherwise.} \end{cases} \tag{13}$$

Each microservice stage is executed by exactly one available replica:

$$\sum_{v \in \mathcal{V}} z_{r,i,v} = 1, \quad \forall r, i, \tag{14}$$

and

$$z_{r,i,v} \le x_{m_i,v}(s(\tau_{r,i})), \quad \forall r, i, v. \tag{15}$$

Let $v_{r,i}$ denote the selected execution satellite satisfying $z_{r,i,v_{r,i}} = 1$.

For routing segment i of request r, let $f_{r,i,e}(t) \ge 0$ be the amount of data carried by link e in slot t. For $i \le L$, the segment routes $B_{r,i}$ from $u_{r,i}$ to $v_{r,i}$; for $i = L+1$, it routes the final output $B_{r,L+1}$ from $v_{r,L}$ to v_d. The link-capacity constraint is

$$0 \le f_{r,i,e}(t) \le R_e(t)\Delta_{r,i}^{\mathrm{rem}}(t), \quad \forall e \in \mathcal{E}(t), \tag{16}$$

where $\Delta_{r,i}^{\mathrm{rem}}(t)$ is the remaining usable time in slot t. If a segment cannot finish within the current slot, ELARA updates the residual data at the slot boundary and re-optimizes routing in the next topology snapshot.

Let $T_{r,i}^{\mathrm{ru}}$ and $E_{r,i}^{\mathrm{ru}}$ denote the routing delay and energy of segment i accumulated over all slots it spans. For service stage $i \le L$, the data arrival time and computation finish time are

$$\tau_{r,i}^{\mathrm{arr}} = \tau_{r,i} + T_{r,i}^{\mathrm{ru}}, \tag{17}$$

and

$$\tau_{r,i}^{\mathrm{fin}} = \tau_{r,i}^{\mathrm{arr}} + T_{m_i}^{\mathrm{cp}}\big(v_{r,i}, s(\tau_{r,i}^{\mathrm{arr}})\big). \tag{18}$$

The request then advances as

$$u_{r,i+1} = v_{r,i}, \quad \tau_{r,i+1} = \tau_{r,i}^{\mathrm{fin}}. \tag{19}$$

After m_L completes, the final output is routed to v_d, and the end-to-end latency is

$$T_r^{\mathrm{e2e}} = \tau_r^{\mathrm{out}} - \tau_{\mathrm{in}}, \tag{20}$$

where τ_r^{out} is the time when the output reaches the destination satellite.

The slow-layer redeployment action in slot t is

$$a_t \in \{\mathrm{add}, \mathrm{remove}, \mathrm{move}, \text{no-op}\}. \tag{21}$$

Every action must preserve (10) and (11). For add and move actions, the target satellite must have enough residual storage and must not already host the same microservice. For remove actions, the service must still have at least $r_m^{\min}$ replicas after removal.

B. Latency, Energy, and Migration Cost

For one unit of data transmitted over link e in slot t, the unit delay follows the decomposition in (5):

$$c_e^T(t) = \frac{1}{R_e(t)} + \tau_e^{\mathrm{pr}}(t) + \tau_e^{\mathrm{qe}}(t) + \tau^{\mathrm{sw}}, \tag{22}$$

where τ^{sw} denotes per-hop switching overhead. The unit communication energy is

$$c_e^E(t) = P_e^{\mathrm{tx}} \frac{1}{R_e(t)}. \tag{23}$$

Let $\phi_e(t)$ be the short-horizon disruption risk of link e, estimated from predictable ISL availability over a small look-ahead window. The edge cost used by the routing layer is

$$c_e(t) = \omega_T c_e^T(t) + \omega_E c_e^E(t) + \omega_R \phi_e(t). \tag{24}$$

For request r, the total routing delay and routing energy are

$$T_r^{\mathrm{ru}} = \sum_{i=1}^{L+1} T_{r,i}^{\mathrm{ru}}, \quad E_r^{\mathrm{ru}} = \sum_{i=1}^{L+1} E_{r,i}^{\mathrm{ru}}. \tag{25}$$

The total computing delay and computing energy are

$$T_r^{\mathrm{cp}} = \sum_{i=1}^{L} \sum_{v \in \mathcal{V}} z_{r,i,v} T_{m_i}^{\mathrm{cp}}\big(v, s(\tau_{r,i}^{\mathrm{arr}})\big), \tag{26}$$

and

$$E_r^{\mathrm{cp}} = \sum_{i=1}^{L} \sum_{v \in \mathcal{V}} z_{r,i,v} E_{m_i}^{\mathrm{cp}}\big(v, s(\tau_{r,i}^{\mathrm{arr}})\big). \tag{27}$$

The total service energy is therefore

$$E_r^{\mathrm{tot}} = E_r^{\mathrm{ru}} + E_r^{\mathrm{cp}}. \tag{28}$$

For replica redeployment action a_t, the migration cost is

$$C^{\mathrm{mig}}(a_t) = \omega_T^{\mathrm{mig}} \frac{T^{\mathrm{mig}}(a_t)}{T_0} + \omega_E^{\mathrm{mig}} \frac{E^{\mathrm{mig}}(a_t)}{E_0}, \tag{29}$$

where $T^{\mathrm{mig}}(a_t)$ and $E^{\mathrm{mig}}(a_t)$ are the delay and energy for transferring the service image. Remove and no-op actions have zero migration-transfer cost.

C. Optimization Objective

Let N_r^{cro} be the number of slot crossings experienced by request r, and let $\mathbb{I}_r^{\mathrm{fail}}$ indicate whether the request fails. The per-request cost is

$$J_r = \omega_T \frac{T_r^{\mathrm{e2e}}}{T_0} + \omega_E \frac{E_r^{\mathrm{tot}}}{E_0} + \omega_S N_r^{\mathrm{cro}} + \omega_F \mathbb{I}_r^{\mathrm{fail}}. \tag{30}$$

Given a sequence of requests $\mathcal{Q}$, the online problem is

$$\mathbf{P1}: \quad \min_{\mathbf{z},\mathbf{f},\mathbf{a}} \quad \sum_{r \in \mathcal{Q}} J_r + \omega_M \sum_{t \in \mathcal{T}} C^{\mathrm{mig}}(a_t) \tag{31a}$$

$$\text{s.t.} \quad (14),\ (15), \tag{31b}$$

$$0 \le f_{r,i,e}(t) \le R_e(t) \Delta_{r,i}^{\mathrm{rem}}(t), \quad \forall r, i, e, t, \tag{31c}$$

$$\sum_{e \in \delta^+(u)} f_{r,i,e}(t) - \sum_{e \in \delta^-(u)} f_{r,i,e}(t) = b_{r,i,u}(t), \quad \forall r, i, u, t, \tag{31d}$$

$$(10),\ (11), \tag{31e}$$

$$a_t \in \{\mathrm{add}, \mathrm{remove}, \mathrm{move}, \text{no-op}\}, \quad \forall t, \tag{31f}$$

$$T_r^{\mathrm{e2e}} \le D_r \quad \text{or} \quad \mathbb{I}_r^{\mathrm{fail}} = 1, \quad \forall r. \tag{31g}$$

In (31d), $b_{r,i,u}(t)$ is the net flow balance for routing hop i of request r: it is positive at the current source, negative at the selected execution or destination node, and zero at relay satellites. The formulation is online because the controller observes the current topology, deployment state, link queues, and compute queues, while future requests and future queue dynamics are unknown.

D. Problem Hardness

Proposition 1. Problem **P1** is NP-hard.

Sketch. Consider a restricted case with a single time slot, fixed replica deployment, fixed effective link rates and compute capacities, and no redeployment. The controller still needs to select one replica for each service in a chain and connect consecutive selected replicas with feasible paths to minimize joint computation and communication cost. This restricted case contains the classical service placement and routing problem as a special case and can encode constrained shortest-path and facility-location-like decisions. Therefore, the restricted problem is NP-hard, and the full problem with cross-slot routing, time-varying queues and rates, and adaptive redeployment is at least as hard.

Solving **P1** exactly is not suitable for online SCPN operation because requests arrive continuously, topology changes every slot, and redeployment decisions have delayed effects. ELARA therefore decomposes the problem into three coupled online layers: a fast PPO-GNN replica selector, a cross-slot min-cost max-flow router, and a slow bandit-based replica redeployment agent.

E. LEO Constellation Model

We consider a general Walker Delta LEO constellation as the space infrastructure of the SCPN. The constellation provides global coverage and is parametrically defined by $\mathcal{W} = (A/P/G, H, I)$, where I is the orbital inclination angle, H is the altitude of the orbits, A is the total number of satellites, P is the number of orbital planes, G is the inter-plane phasing factor, and the orbital phase difference between adjacent planes of $360G/A$ degrees. Typically, LEO satellites are uniformly distributed across the P planes, with each plane containing $N = A/P$ evenly-spaced satellites. We denote the set of satellites by $\mathcal{V} = \{v_{p,n} \mid p \in \{1, \dots, P\}, n \in \{1, \dots, N\}\}$, where $v_{p,n}$ uniquely identifies the n-th satellite in the p-th orbital plane.

To render the continuous orbital dynamics tractable, we divide the constellation period into a sequence of quasi-static time slots, $\mathcal{T} = \{1, 2, \dots, T\}$. Within time slot t, we assume that the constellation topology remains constant so that the feasible ISLs and channel capacity can be approximately constant, too.

To formalize the orbital geometry, the real-time position of satellite $v_{p,n}$ can be represented in spherical coordinates as $(H + R_E, \mho_p, \nu_{p,n}(t))$, where R_E is the Earth's radius, $\mho_p$ is the right ascension of the ascending node (RAAN) of the p-th plane, and $\nu_{p,n}(t)$ denotes the orbital phase at time slot t.

F. ISL Communication Model

The satellite network topology at time slot t is represented by a dynamic directed graph $\mathcal{G}(t) = \big(\mathcal{V}, \mathcal{E}(t)\big)$, where $\mathcal{E}(t)$ is the set of feasible ISLs. In this paper, we assume that each satellite is equipped with four laser optical terminals: two positioned along the roll axis for intra-plane ISLs, and two along the pitch axis for cross-plane ISLs.

a) ISL Connectivity Constraints: Intra-plane ISLs are connected by a satellite $v_{p,n}$ to its adjacent neighbors v_{p,n^-} and v_{p,n^+} within the same orbital plane, where $n^{\pm} = ((n \pm 1 - 1) \bmod N) + 1$. Since satellites within the same plane share identical orbital velocities, their relative physical distances remain invariant, forming highly stable ISLs with each other.

Conversely, cross-plane ISLs connecting satellites in adjacent orbital planes are inherently dynamic. To encapsulate the physical and kinematic limitations of the onboard laser communication terminals (LCTs), the topological connectivity of a cross-plane ISL (u, v) is governed by three joint constraints: line-of-sight (LoS) occlusion, mechanical tracking limits, and polar exclusion. The existence of a cross-plane link at time slot t can be formally expressed as:

$$(u,v) \in \mathcal{E}(t) \iff \begin{cases} d_{u,v}(t) \le d_{\max}, \\ \theta_{u,v}(t) \le \theta_{\max}, \\ \max\big(|\varphi_u(t)|, |\varphi_v(t)|\big) \le \varphi_{\max}. \end{cases} \tag{32}$$

In (32), the dynamic LoS distance $d_{u,v}(t)$ is derived in spherical coordinates as:

$$\begin{aligned} d_{u,v}(t) = \Big(2(H+R_E)^2\big[1 - \cos\nu_u(t)\cos\nu_v(t) \\ - \cos(\mho_p - \mho_q)\sin\nu_u(t)\sin\nu_v(t)\big]\Big)^{\frac{1}{2}}, \end{aligned} \tag{33}$$

where $d_{\max} = 2\sqrt{H(H+2R_E)}$ represents the theoretical maximum distance before terrestrial occlusion. The variable $\theta_{u,v}(t)$ denotes the relative tracking angle, which is bounded by the maximum tolerable hardware threshold $\theta_{\max}$. The third constraint enforces a polar exclusion zone parameterized by a latitude boundary $\varphi_{\max}$ (e.g., 70°). This mechanism disconnects satellites entering polar regions, mitigating the severe pointing dynamics and extreme Doppler shifts inherent to high-inclination orbits.

b) ISL Capacity Model: Unlike ground networks with stable capacities, the transmission rate of laser ISLs is inherently distance-dependent due to geometric propagation loss. Under the Free-Space Optical (FSO) communication model, the received optical power $P^{\mathrm{rx}}_{u,v}(t)$ at satellite v transmitted from satellite u is formalized by the Friis transmission equation variant:

$$P^{\mathrm{rx}}_{u,v}(t) = P^{\mathrm{tx}} G^{\mathrm{tx}} G^{\mathrm{rx}} \eta_{\mathrm{tx}} \eta_{\mathrm{rx}} \left(\frac{\lambda}{4\pi d_{u,v}(t)}\right)^2, \tag{34}$$

where P^{tx} is the nominal laser transmit power, G^{tx} and G^{rx} represent the optical gains of the transmitter and receiver apertures respectively, λ is the laser wavelength, and $\eta_{\mathrm{tx}}, \eta_{\mathrm{rx}}$ denote the optical efficiency factors.

Consequently, the real-time Signal-to-Noise Ratio (SNR) of the link (u, v) at time slot t is formulated as:

$$\gamma_{u,v}(t) = \frac{\big(R_{\mathrm{pd}} \cdot P^{\mathrm{rx}}_{u,v}(t)\big)^2}{\sigma^2_{\mathrm{sh}} + \sigma^2_{\mathrm{th}}}, \tag{35}$$

where R_{pd} is the photodiode responsivity, while σ^2_{sh} and σ^2_{th} denote the shot and thermal noise variances, respectively. According to the Shannon-Hartley theorem, the achievable transmission capacity $R_{u,v}(t)$ (in bits per second) is constrained by:

$$R_{u,v}(t) = B_{u,v} \log_2\left(1 + \gamma_{u,v}(t)\right), \tag{36}$$

where $B_{u,v}$ is the modulation bandwidth.

G. Microservice Request Model

To facilitate flexible application management in the SCPN, software capabilities are decoupled into a set of fine-grained, loosely coupled microservices, denoted by $\mathcal{M} = \{m_1, m_2, \dots, m_M\}$. Each microservice m_i exhibits a computational demand c_i (in CPU cycles) and produces an intermediate data payload $D_{i,i+1}$ (in bits), which must be transferred to the location of m_{i+1}. To accommodate severe onboard storage constraints, microservices are selectively cached. The binary indicator $x_{i,v} = 1$ if a replica of m_i is deployed on satellite v. The system leverages a highly

compressed service replica directory $\mathcal{D}_{\text{svc}} : m_i \mapsto \mathcal{V}_{m_i}$ to efficiently locate candidates without full network queries.

We model user workloads as Service Function Chains (SFCs). Specifically, a service request is formally abstracted as a directed, time-constrained sequence $\mathcal{C} = (m_1 \to m_2 \to \cdots \to m_L)$, where m_i denotes the i-th dependent microservice, L is the chain length, and the entire execution process should be completed within a predefined deadline $T_{\max}$.

H. Onboard Computing and Energy Model

Each satellite $v \in \mathcal{V}$ provides a computational processing capacity f_v. Due to the concurrency of incoming requests, satellites experience dynamic load fluctuations. Let $q_v(t)$ denote the equivalent queued workload (in CPU cycles) at satellite v in slot t. If microservice m_i is routed to v, the holistic computation delay is formulated as:

$$T_i^{\text{cp}}(v,t) = \frac{q_v(t) + c_i}{f_v}, \tag{37}$$

which captures both the execution time and the waiting time in the queue. Concurrently, the computational energy consumed by the processor on satellite v executing microservice m_i is modeled as

$$E_v^{\text{cp}}(i,t) = P_v^{\text{cp}} \frac{c_i}{f_v} \tag{38}$$

where P_v^{cp} is the average active working power of the payload.

I. Communication and Service Subflow and Model

We model the communication process between satellites as consisting of two fundamental components: transmission delay and propagation delay. For an arbitrary active ISL $e = (u,v) \in \mathcal{E}(t)$ connecting two adjacent satellites at time slot t, the transmission delay incurred to send a data payload of D bits is, $T_e^{\text{tr}}(D,t) = D/R_e(t)$, and the spatial propagation delay is formulated as, $T_e^{\text{pr}}(t) = d_e(t)/c$, where $d_e(t) = d_{u,v}(t)$ is the physical distance between satellites u and v, and c is the speed of light.

During the onboard SFC execution, consecutive microservices are often hosted on geographically dispersed satellites, necessitating multi-hop routing. Moreover, transferring massive intermediate data payloads—or initiating transmissions near topological handovers—often causes the communication to span multiple time slots. To ensure robust data delivery against such volatility, establishing multi-path inter-satellite connections is imperative.

To resiliently route the intermediate data $D_{i,i+1}$ from source satellite s (hosting m_i) to target satellite d (hosting m_{i+1}) in time slot t, we maintain a feasible candidate path set $\mathcal{P}_{s,d}(t) = \{p_{s,d}^1(t), \ldots, p_{s,d}^K(t)\}$, where K paths are retained for online decision-making. For notational brevity, we simplify $\mathcal{P}_{s,d}(t)$ to $\mathcal{P}_i(t)$ and its k-th element to $p_i^k(t)$. This candidate path set can be generated using the geometry-aware algorithm proposed in [37], which incurs only an $\mathcal{O}(1)$ computational complexity.

However, due to the orbital dynamics, ISL paths are vulnerable to transitions across time slots (e.g., satellite enters the polar zone). We formally define the residual lifetime $L_e(t)$ of link e as the remaining time duration before the link state deteriorates. Consequently, the residual lifetime of an entire path p evaluated at slot t is strictly constrained by its most fragile segment:

$$L_p(t) = \min_{e \in p} L_e(t). \tag{39}$$

As for the multipath routing decision, we define a binary path selection indicator $\xi_i^k \in \{0,1\}$ and a continuous data-splitting fraction $\lambda_i^k \in [0,1]$. These variables adhere to the flow conservation constraints:

$$\sum_{k=1}^{K_i} \lambda_i^k = 1, \quad 0 \le \lambda_i^k \le \xi_i^k, \quad \forall k. \tag{40}$$

This framework supports flexible transmission paradigms governed by the combination of ξ_i^k and λ_i^k:

A critical challenge in LEO networks is that data transmissions may span multiple time slots, due to substantial data volumes and variable transmission initiation times. Such cross-slot transmissions inevitably suffer from rate degradation, TCP retransmissions, and forced topological handovers. To rigorously model this cross-slot effect, the effective end-to-end communication delay over a chosen subflow k incorporates the nominal transmission delay, the aggregated propagation delay $T_{p_i^k}^{\text{pr}}(t)$, and a cross-slot handover delay penalty $\Gamma_{i,k}^{\text{delay}}(t)$. The subflow delay is formulated as:

$$T_{i,k}^{\text{tr}}(t) = \frac{\lambda_i^k D_{i,i+1}}{R_{p_i^k}(t)} + T_{p_i^k}^{\text{pr}}(t) + \Gamma_{i,k}^{\text{delay}}(t), \tag{41}$$

where the penalty $\Gamma_{i,k}^{\text{delay}}(t) = \alpha \cdot \max\left(0, \frac{\lambda_i^k D_{i,i+1}}{R_{p_i^k}(t)} - L_{p_i^k}(t)\right)$ explicitly penalizes routing decisions that unsafely cross slot boundaries, with α representing the delay overhead coefficient. The overall communication delay for the service dependency is thus bounded by the slowest active subflow, augmented by the multi-path coordination overhead ω_i^{mp}:

$$T_i^{\text{tr}}(t) = \max_{k:\xi_i^k=1} T_{i,k}^{\text{tr}}(t) + \omega_i^{\text{mp}}. \tag{42}$$

Similarly, if a subflow spans across time slots, additional energy is dissipated due to control-plane signaling and link re-synchronization. The aggregate communication energy $E_i^{\text{tr}}(t)$ is formulated by summing the active link expenditures, the cross-slot energy penalty $\Gamma_{i,k}^{\text{energy}}(t)$, and the macroscopic multipath signaling overhead Ω_i^{mp}:

$$E_i^{\text{tr}}(t) = \sum_{k=1}^{K_i} \xi_i^k \left(\sum_{e \in p_i^k(t)} P_e^{\text{tr}} \frac{\lambda_i^k D_{i,i+1}}{R_e(t)} + \Gamma_{i,k}^{\text{energy}}(t) \right) + \Omega_i^{\text{mp}}, \tag{43}$$

where P_e^{tr} is the nominal transmit power over link e, and the energy penalty $\Gamma_{i,k}^{\text{energy}}(t)$ follows a similar $\max(\cdot,0)$ piecewise structure to penalize temporal overruns.

V. Problem Formulation

This section formulates the energy-first, latency-constrained microservice routing problem. The system jointly decides the execution satellite for each microservice, the service subflow paths between consecutive microservices, and the data-splitting ratios over selected paths. The primary optimization objective is to minimize total energy consumption, while the end-to-end service latency must satisfy a request-level deadline.

A. Decision Variables

For each microservice m_i and satellite v, we define the execution placement variable

$$z_{i,v} = \begin{cases} 1, & \text{if } m_i \text{ is executed on satellite } v, \\ 0, & \text{otherwise.} \end{cases} \tag{44}$$

Each microservice must be executed by exactly one satellite hosting its replica:

$$\sum_{v \in \mathcal{V}_{m_i}} z_{i,v} = 1, \quad \forall i = 1, \ldots, L, \tag{45}$$

and

$$z_{i,v} = 0, \quad \forall v \notin \mathcal{V}_{m_i}. \tag{46}$$

For each microservice dependency $m_i \rightarrow m_{i+1}$, we define path selection variables $\{y_i^k\}_{k=1}^{K_i}$ and data-splitting variables $\{\lambda_i^k\}_{k=1}^{K_i}$ as in Section ??. The selected paths must satisfy

$$y_i^k \in \{0,1\}, \quad \forall i, k, \tag{47}$$

$$0 \le \lambda_i^k \le y_i^k, \quad \forall i, k, \tag{48}$$

and

$$\sum_{k=1}^{K_i} \lambda_i^k = 1, \quad \forall i = 1, \ldots, L-1. \tag{49}$$

B. End-to-End Latency

The computation delay of microservice m_i is

$$T_i^{\mathrm{cp}} = \sum_{v \in \mathcal{V}_{m_i}} z_{i,v} T_i^{\mathrm{cp}}(v, s_i), \tag{50}$$

where s_i denotes the slot in which m_i is executed.

The communication delay between m_i and m_{i+1} is given by (42). Since we consider non-pipelined chain execution, m_{i+1} can start only after m_i has completed its computation and the corresponding intermediate data has arrived. Thus, the end-to-end service latency is

$$T^{\mathrm{e2e}} = \sum_{i=1}^{L} T_i^{\mathrm{cp}} + \sum_{i=1}^{L-1} T_i^{\mathrm{tr}}. \tag{51}$$

The request is associated with a maximum tolerable latency $T^{\max}$, and therefore must satisfy

$$T^{\mathrm{e2e}} \le T^{\max}. \tag{52}$$

C. Energy Consumption

The total computation energy is

$$E^{\mathrm{cp}} = \sum_{i=1}^{L} \sum_{v \in \mathcal{V}_{m_i}} z_{i,v} E_i^{\mathrm{cp}}(v, s_i). \tag{53}$$

The total communication energy is

$$E^{\mathrm{tr}} = \sum_{i=1}^{L-1} E_i^{\mathrm{tr}}(s_i), \tag{54}$$

where $E_i^{\mathrm{tr}}(s_i)$ is defined in (43). The overall energy consumption for completing the chain request is

$$E^{\mathrm{total}} = E^{\mathrm{cp}} + E^{\mathrm{tr}}. \tag{55}$$

D. Path-Switching Risk and Battery-Aware Penalty

For a selected path $p_i^k(s_i)$, if the estimated transmission delay is close to or exceeds the residual path lifetime, the transmission may be interrupted by a topology transition. We define the path-switching risk as

$$\Phi_i^{\mathrm{risk}} = \sum_{k=1}^{K_i} y_i^k \exp\left(-\frac{L_{p_i^k}(s_i)}{T_{i,k}^{\mathrm{tr}}(s_i) + \epsilon} \right), \tag{56}$$

where ϵ is a small positive constant to avoid division by zero.

The total battery penalty is

$$\Phi^{\mathrm{bat}} = \sum_{i=1}^{L} \sum_{v \in \mathcal{V}_{m_i}} z_{i,v} \Psi_v^{\mathrm{bat}}(s_i). \tag{57}$$

The control overhead caused by candidate probing and multipath maintenance is modeled as

$$\Phi^{\mathrm{ctrl}} = \sum_{i=1}^{L-1} \left(c_0 |\mathcal{C}_{i+1}^K| + c_1 \sum_{k=1}^{K_i} y_i^k \right), \tag{58}$$

where c_0 and c_1 are weighting coefficients for probing overhead and path-maintenance overhead, respectively.

E. Energy-First Latency-Constrained Optimization

The optimization objective is energy-first. Specifically, the system minimizes the total energy consumption, while latency is treated as a secondary optimization term and

is also bounded by a hard deadline. The problem is formulated as

$$\mathbf{P1}: \quad \min_{\mathbf{z},\mathbf{y},\boldsymbol{\lambda}} \quad \frac{E^{\text{total}}}{E_0} + \varepsilon_T \frac{T^{\text{e2e}}}{T_0} + \omega_R \Phi^{\text{risk}} + \omega_B \Phi^{\text{bat}} + \omega_C \Phi^{\text{ctrl}} \tag{59a}$$

$$\text{s.t.} \quad \sum_{v \in \mathcal{V}_{m_i}} z_{i,v} = 1, \quad \forall i, \tag{59b}$$

$$z_{i,v} = 0, \quad \forall v \notin \mathcal{V}_{m_i}, \ \forall i, \tag{59c}$$

$$z_{i,v} \in \{0,1\}, \quad \forall i, v, \tag{59d}$$

$$y_i^k \in \{0,1\}, \quad \forall i, k, \tag{59e}$$

$$0 \le \lambda_i^k \le y_i^k, \quad \forall i, k, \tag{59f}$$

$$\sum_{k=1}^{K_i} \lambda_i^k = 1, \quad \forall i = 1, \ldots, L-1, \tag{59g}$$

$$T^{\text{e2e}} \le T^{\max}, \tag{59h}$$

$$B_v(s_i) \ge B_v^{\min}, \quad \forall v, i, \tag{59i}$$

$$\lambda_i^k > 0 \Rightarrow p_i^k(s_i) \in \mathcal{P}_i(s_i), \quad \forall i, k. \tag{59j}$$

Here, E_0 and T_0 are normalization constants, ε_T is a small coefficient satisfying $\varepsilon_T \ll 1$, and ω_R, ω_B, and ω_C are non-negative coefficients. The small coefficient ε_T ensures that energy remains the primary objective, while latency is optimized secondarily under the hard deadline constraint in (59h).

Constraint (59b) ensures that each microservice is executed by exactly one satellite replica. Constraint (59c) guarantees that a microservice can only be executed on satellites where it is deployed. Constraints (59e)–(59g) define feasible service subflow selection and data splitting. Constraint (59h) imposes the end-to-end latency requirement. Constraint (59i) prevents energy-critical satellites from being selected. Constraint (59j) ensures that data is transmitted only through feasible paths in the corresponding topology slot.

F. Discussion on Problem Complexity

Problem **P1** is a mixed-integer nonlinear optimization problem. The binary variables $\mathbf{z}$ determine microservice replica selection, while $\mathbf{y}$ and $\boldsymbol{\lambda}$ determine service subflow selection and data splitting. The coupling among computation delay, communication delay, path lifetime, battery states, and time-varying topology makes the problem challenging to solve optimally in real time. Moreover, the satellite load and battery states are only available through lightweight probing and may change during the request execution process.

Therefore, rather than solving **P1** globally with full future knowledge, we design an online energy-aware decision scheme. The key idea is to exploit predictable orbital dynamics for offline topology indexing, while using lightweight online feedback to adaptively select the next execution satellite and transmission mode. Specifically, the proposed solution consists of two coupled components:

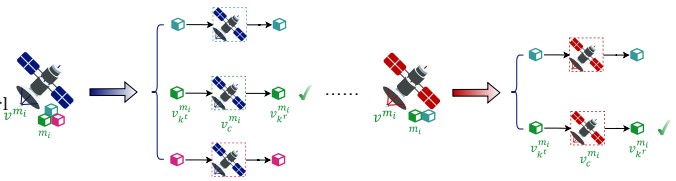


Fig. 2: Abstract the computation process from colocated microservice and satellite deployment.

a contextual-bandit-based replica selection module and an MPTCP-inspired adaptive path selection module.

VI. Energy-aware Service Routing Algorithm for Microservices in the SCPN

In this section, we present our approach to addressing the proposed problem P as outlined in Section IV. We first detail our augmented satellite-microservice connected subgraph construction algorithm, and then elaborate on the design of our DST-based service routing algorithm tailored to optimize energy consumption in the SCPN.

A. Augmented Satellite-Microservice Sub-Graph Construction

For each request originating from a terrestrial station, there is a DAG $\mathcal{G}_f^m$, which outlines the service execution sequence. In $\mathcal{G}_f^m$, there may exist ρ ($1 \le \rho \le |\mathcal{M}_f| - 1$) vertices whose outdegree equals 0. These vertices represent terminal microservices that conclude the processing sequence by dispatching results to their respective destinations. Consequently, there should be multiple logical destination satellite nodes, denoted as $\{v^{dst_1}, \ldots, v^{dst_\rho}\}$. The routing paths from the logical destination satellites hosting the terminal microservices to the real destination satellites can be straightforwardly generated by the control panel. Note that if a terminal microservice is deployed directly on its real destination satellite, this satellite also acts as the logical destination. In such cases, the communication cost between the terminal service and its destination is effectively zero. Next, we define the augmented graph to be constructed as $\mathcal{G}_z = (\mathcal{V}_z, \mathcal{E}_z)$, where initially, $\mathcal{V}_z = \varnothing$ and $\mathcal{E}_z = \varnothing$. Then, the process of constructing graph $\mathcal{G}_z$ is detailed as follows:

Step ❶. This step involves adding the source node v^{src}, along with the logical destination nodes $\{v^{dst_1}, \ldots, v^{dst_\rho}\}$ and the real destination node v_{dst}, to vertex collection $\mathcal{V}_z$.

Step ❷. For each microservice m_i in $\mathcal{M}_f$, and $\forall v_k \in \mathcal{V}_{m_i}$, this step creates three distinct vertices: $v_{k^r}^{m_i}$, $v_{k^c}^{m_i}$ and $v_{k^t}^{m_i}$, which are then added to the set $\mathcal{V}_z$. Subsequently, it constructs two directed edges $v_{k^r}^{m_i} \to v_{k^c}^{m_i}$ and $v_{k^c}^{m_i} \to v_{k^t}^{m_i}$, both with weights equal to $E_{m_i}(v_k)/2$. These edges are added to the edge set $\mathcal{E}_z$. This procedure is detailed in Fig. 2.

Step ❸. For the calling relationship between two microservices m_i and m_j, that is, $\forall w_{i,j} \in \mathcal{W}_f$, the algorithm will iteratively identify all satellite nodes from collection $\mathcal{V}$ that host microservices m_i and m_j, respectively. If

Algorithm 1: Augmented Satellite-Microservice Connected Subgraph Construction Algorithm.

Initialize augmented directed graph $\mathcal{G}_z$ with vertex set $\mathcal{V}_z \leftarrow \varnothing$ and edge set $\mathcal{E}_z \leftarrow \varnothing$;
$\mathcal{V}_z \leftarrow \mathcal{V}_z \bigcup \{v^{src}, v^{dst_1}, \ldots, v^{dst_\rho}\}$;
for m_i in $\mathcal{M}_f$ do
  for v_k in $\mathcal{V}_{m_i}$ do
    $\mathcal{V}_z \leftarrow \mathcal{V}_z \bigcup \{v_{k^r}^{m_i}, v_{k^c}^{m_i}, v_{k^t}^{m_i}\}$;
    $\mathcal{E}_z \leftarrow \mathcal{E}_z \bigcup \{(v_{k^r}^{m_i} \rightarrow v_{k^c}^{m_i}), (v_{k^c}^{m_i} \rightarrow v_{k^t}^{m_i})\}$;
    Set $\mathbf{w}(v_{k^r}^{m_i} \rightarrow v_{k^c}^{m_i}) \doteq E_{m_i}(v_k)/2$;
    Set $\mathbf{w}(v_{k^c}^{m_i} \rightarrow v_{k^t}^{m_i}) \doteq E_{m_i}(v_k)/2$;
  end
end
for $w_{i,j}$ in $\mathcal{W}_f$ do
  for $v_k^{m_i}$ in $\mathcal{V}_{m_i}$ do
    for $v_k^{m_j}$ in $\mathcal{V}_{m_j}$ do
      $\mathcal{V}_o^{re}, \mathcal{E}_o^{re} = \text{GISRSA}(\mathcal{G}, v_k^{m_i}, v_k^{m_j})$;
      $\mathcal{V}_z \leftarrow \mathcal{V}_z \bigcup \mathcal{V}_o^{re}$;
      $\mathcal{E}_z \leftarrow \mathcal{E}_z \bigcup \mathcal{E}_o^{re}$;
    end
  end
end
for $v_k^{m_{src}}$ in $\mathcal{V}_{m_{src}}$ do
  $\mathcal{V}_o^{re}, \mathcal{E}_o^{re} = \text{GISRSA}(\mathcal{G}, v^{src}, v_k^{m_{src}})$;
  $\mathcal{V}_z \leftarrow \mathcal{V}_z \bigcup \mathcal{V}_o^{re}$;
  $\mathcal{E}_z \leftarrow \mathcal{E}_z \bigcup \mathcal{E}_o^{re}$;
end
for $v_k^{m_{dst}}$ in $\mathcal{V}_{m_{dst}}$ do
  for g in $[1, \rho]$ do
    $\mathcal{V}_o^{re}, \mathcal{E}_o^{re} = \text{GISRSA}(\mathcal{G}, v_k^{m_{dst}}, V^{dst_g})$;
    $\mathcal{V}_z \leftarrow \mathcal{V}_z \bigcup \mathcal{V}_o^{re}$;
    $\mathcal{E}_z \leftarrow \mathcal{E}_z \bigcup \mathcal{E}_o^{re}$;
  end
end

satellites $v_k^{m_i}$ and $v_k^{m_j}$ are directly adjacent without needing any relay satellite to facilitate the communication, then a directed edge $v_{k^r}^{m_i} \rightarrow v_{k^t}^{m_j}$ is constructed, and its weight is set as the communication energy cost from $v_{k^r}^{m_i}$ to $v_{k_t}^{m_j}$ in $\mathcal{G}$, given as:

$$\mathbf{e}^{tr} = P_T^{v_{k^r}^{m_i} \rightarrow v_{k^t}^{m_j}} \frac{w_{i,j}}{\mathrm{w}(v_{k^r}^{m_i} \rightarrow v_{k^t}^{m_j})}, \tag{60}$$

Otherwise, the algorithm then searches for available relay satellite nodes within $\mathcal{V}$, which can be efficiently resolved using the Floyd algorithm, resulting in a collection of relay satellite nodes denoted as $\mathcal{V}^{re} = \{v_1^{re}, \ldots v_\xi^{re}\}$. Then, it adds these relay nodes to $\mathcal{V}_z$, and constructs and adds edges $v_{k^t}^{m_i} \rightarrow v_a^{re}, v_a^{re} \rightarrow v_{a+1}^{re}, v_\xi^{re} \rightarrow v_{k^r}^{m_j}, (\forall a \in [1, \xi - 1])$ to $\mathcal{E}_z$. The weights of these edges, representing the relay communication energy cost between satellites, are calculated according to the unit-energy model in Eq. 23, as

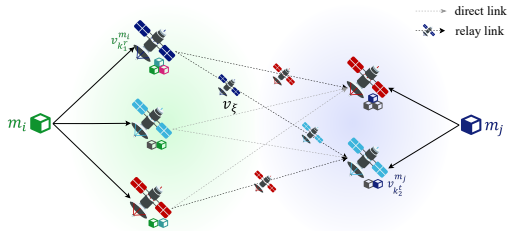


Fig. 3: Build the relay routing paths for neighbor microservices that are deployed on non-adjacent satellites.

below:

$$\mathbf{e}_{re_1}^{tr}(v_{k^t}^{m_i} \rightarrow v_a^{re}) = P_T^{v_{k^t}^{m_i} \rightarrow v_a^{re}} \frac{w_{i,j}}{\mathrm{w}(v_{k^t}^{m_i} \rightarrow v_a^{re})}, \tag{61}$$

$$\mathbf{e}_{re_2}^{tr}(v_a^{re} \rightarrow v_{a+1}^{re}) = P_T^{v_a^{re} \rightarrow v_{a+1}^{re}} \frac{w_{i,j}}{\mathrm{w}(v_a^{re} \rightarrow v_{a+1}^{re})}, \tag{62}$$

$$\mathbf{e}_{re_3}^{tr}(v_\xi^{re} \rightarrow v_{k^r}^{m_j}) = P_T^{v_\xi^{re} \rightarrow v_{k^r}^{m_j}} \frac{w_{i,j}}{\mathrm{w}(v_\xi^{re} \rightarrow v_{k^r}^{m_j})}. \tag{63}$$

This process is depicted in Fig . 3.

Step ❹. For the first executed microservice m_{src} and the final executed microservices $\{m_{dst^1}, \ldots, m_{dst^\rho}\}$, the algorithm searches for the collection of satellite nodes hosting these microservices. Subsequently, it constructs the edges $v^{src} \rightarrow v_{k^r}^{m_{src}}$ and $v_{k^t}^{m_{dst}} \rightarrow v^{dst_g}, \forall g \in [1, \rho]$. The detailed procedures for the construction of the newly generated vertices and edges are similar to those outlined in Step ❸.

We summarize the above steps in Algorithm 1. This systematic approach guarantees that within the augmented graph $\mathcal{G}_z$, each tree rooted at the source node v^{src}, which includes the destination node v^{dst} and all $v_{k^c}^{m_i}$ nodes, constitutes a feasible service routing scheme ψ. This scheme ensures that all computation and communication processes required in $\mathcal{G}_{f'}^m$ are executed sequentially, and the sum of the weights of all edges in this tree quantifies the total energy E^{total} required for the routing scheme. Subsequently, solving the original problem can essentially be transformed into tackling a DST problem on the graph $\mathcal{G}_z$. The objective is to identify a connected subgraph that contains all specified nodes in graph $\mathcal{G}_z$ such that the sum of the weights of all edges in the subgraph is minimized. The directed Steiner tree problem is defined as follows.

Definition 4. (Directed Steiner Tree). Given a directed graph G=(V, A), an assigned root node $r \in$ V, and a set of terminals X $\subseteq$ V, (|X| = k), the objective is to find the minimum cost arborescence rooted at r and spanning all the vertices in X.

B. DST-based Service Routing Algorithm

The DST problem is known to be NP-hard, and extensive research has been conducted on it. Numerous studies have proposed approximation solutions for the original problem [29], [30]. However, solving the DST problem within the LEO network topology introduces unique and particularly severe challenges due to the dynamic nature

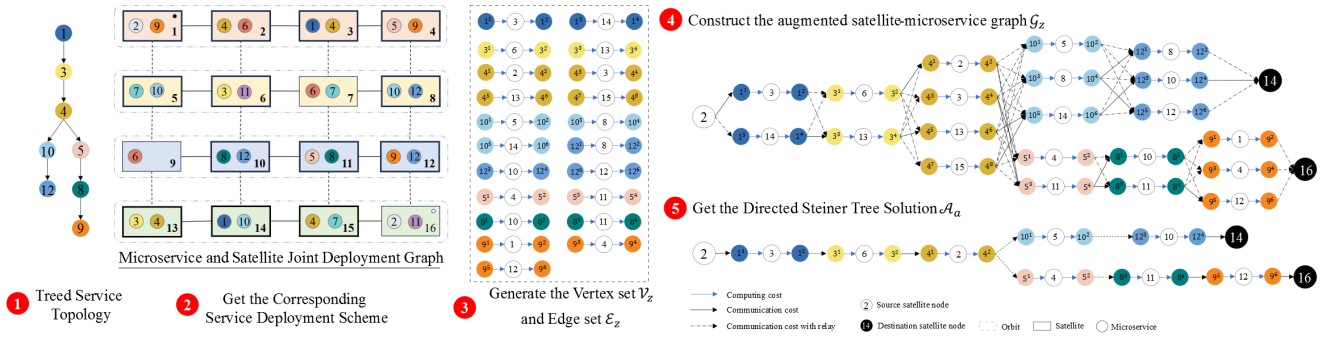


Fig. 4: The overview of solution workflow of tree-based request topology.

Algorithm 2: Approximated Solution for Minimized Spanning Directed Steiner Arborescence.

Input : $\mathcal{G}_z, \{v^{src}, v^{dst_1}, \dots v^{dst_\rho}\}$
Output: $\mathcal{A}(\mathcal{V}_\mathrm{a}, \mathcal{E}_\mathrm{a})$

Init the terminal pairs set, $\{(v^{src}, v^{dst_\mathrm{g}}) | \forall \mathrm{g} \in [1, \rho]\}$;
Init $\mathcal{V}_\mathrm{a} \leftarrow \varnothing, \mathcal{E}_\mathrm{a} \leftarrow \varnothing$;
for g in $[1, \rho]$ do
  $\mathcal{V}_g, \mathcal{E}_g$ = Dijkstra$(\mathcal{G}_z, v^{src}, v^{dst_\mathrm{g}})$
end
$\mathcal{V}_\mathrm{a} \leftarrow \bigcap_{\mathrm{g}=1}^{\rho} \mathcal{V}_\mathrm{g}$;
$\mathcal{E}_\mathrm{a} \leftarrow \mathcal{E}_1$;
for g in $[2, \rho]$ do
  for e in $\mathcal{E}_\mathrm{g}$ do
    if $e \in \mathcal{E}_\mathrm{a}$ then
      $\mathrm{w}(e) \doteq \min(\mathrm{w}(\mathcal{E}_\mathrm{a}^e), \mathrm{w}(\mathcal{E}_\mathrm{g}^e))$
    else
      $\mathcal{E}_\mathrm{a} \leftarrow \mathcal{E}_\mathrm{a} \cup \{e\}$
    end
  end
end
Construct $\mathcal{A}(\mathcal{V}_\mathrm{a}, \mathcal{E}_\mathrm{a})$;

Algorithm 3: Generate the Inter-Satellite Relay Routing Scheme Algorithm (GISRA).

Input : $\mathcal{G}, v_k^{m_i}, v_k^{m_j}$
Output: $\mathcal{V}_o^{re}, \mathcal{E}_o^{re}$

Initialize the vertex set and edge set with $\mathcal{V}_o^{re} \leftarrow \varnothing, \mathcal{E}_o^{re} \leftarrow \varnothing$;
if $v_k^{m_i}$ is adjacent to $v_k^{m_j}$ then
  $\mathcal{E}_o^{re} \leftarrow \mathcal{E}_z \bigcup \{(v_{k^r}^{m_i} \rightarrow v_{k^t}^{m_j})\}$;
  $\mathbf{w}(v_{k^r}^{m_i} \rightarrow v_{k^t}^{m_j}) \doteq \mathbf{e}^{tr}(v_{k^r}^{m_i} \rightarrow v_{k^t}^{m_j})$;
else
  $\mathcal{V}_o^{re}$, path = Floyd$(\mathcal{G}, v_k^{m_i}, v_k^{m_j})$;
  $\mathcal{V}_o^{re} \leftarrow \mathcal{V}_o^{re} \setminus \{v_k^{m_i}, v_k^{m_i}\}$;
  $\mathcal{E}_o^{re} \leftarrow \mathcal{E}_o^{re} \bigcup \{(v_{k^t}^{m_i} \rightarrow v_1^{re}), (v_\xi^{re} \rightarrow v_{k^r}^{m_j})\}$;
  $\mathbf{w}(v_{k^t}^{m_i} \rightarrow v_1^{re}) \doteq \mathbf{e}_{re_1}^{tr}(v_{k^t}^{m_i} \rightarrow v_1^{re})$;
  $\mathbf{w}(v_\xi^{re} \rightarrow v_{k^r}^{m_j}) \doteq \mathbf{e}_{re_3}^{tr}(v_\xi^{re} \rightarrow v_{k^r}^{m_j})$;
  for $a \in [1, \xi - 1]$ do
    $\mathcal{V}_o^{re} \leftarrow \mathcal{V}_o^{re} \bigcup \{v_a^{re}, v_{a_1}^{re}\}$;
    $\mathcal{E}_o^{re} \leftarrow \mathcal{E}_o^{re} \bigcup \{(v_a^{re} \rightarrow v_{a_1}^{re})\}$;
    $\mathbf{w}(v_a^{re} \rightarrow v_{a_1}^{re}) \doteq \mathbf{e}_{re_2}^{tr}(v_a^{re} \rightarrow v_{a_1}^{re})$;
  end
end

of satellite constellations. Specifically, the time-varying nature of ISLs owing to the constant movement of satellites results in a highly dynamic network topology, making the link connectivity a complex issue. LoS constraints further complicate the ability to guarantee consistent connections between terminals. Additionally, microservices that need to communicate may not be colocated on the same or directly adjacent satellites, necessitating the use of relay satellites to establish communication paths. These challenges motivate us to design a specialized heuristic algorithm tailored to the ever-changing LEO satellite networks.

The main steps of our energy-aware DST-based service routing algorithm are as follows:

1) Augmented Graph Initialization: In time slot s, the augmented graph $\mathcal{G}_z(\mathrm{s}) = (\mathcal{V}_z(\mathrm{s}), \mathcal{E}_z(\mathrm{s}))$ is initialized with weighed edges.

2) Get the Shortest Path: For each root and terminal node pairs $(v^{src}, v^{dst_\mathrm{g}}), \forall \mathrm{g} \in [1, \rho]$, it recursively finds the minimum cost path by utilizing the Dijkstra algorithm. Record the involved vertices and edges that constitute each path, represented by $\{\mathcal{V}_1(\mathrm{s}), \dots, \mathcal{V}_\rho(\mathrm{s})\}$ and $\{\mathcal{E}_1(\mathrm{s}), \dots, \mathcal{E}_\rho(\mathrm{s})\}$.

3) Merge the Shortest Paths: Aggregate the vertex sets from the minimum cost paths and remove any duplicates to obtain $\mathcal{V}_\mathrm{a}(\mathrm{s}) = \bigcap_{\mathrm{g}=1}^{\rho} \mathcal{V}_\mathrm{g}(\mathrm{s})$. Then, consolidate these edge sets to $\mathcal{E}_\mathrm{a}(\mathrm{s}) = \bigcap_{\mathrm{g}=1}^{\rho} \mathcal{E}_\mathrm{g}(\mathrm{s})$. Note that if any edges share common vertices, select the edge with the smaller weight to ensure the most cost-effective connections.

4) Minimum Spanning Steiner Arborescence: Utilize the aggregated vertex and edge collections to construct the minimum spanning Steiner arborescence for the snapshot s, denoted by $\mathcal{A}(\mathrm{s}) = (\mathcal{V}_\mathrm{a}(\mathrm{s}), \mathcal{E}_\mathrm{a}(\mathrm{s}))$. This step provides an approximate solution to the DST problem, optimizing the cost while ensuring connectivity among specified nodes.

Algorithm 2 presents a detailed step-by-step procedure for the proposed approach, offering a comprehensive

view of its implementation. To enhance understanding, a simplified example is provided in Fig. 4, which visually demonstrates the key steps and logic of the algorithm.

C. Algorithm Complexity Analysis

The complexity of Algorithm 1 and 2 is analyzed as follows.

Complexity of Algorithm 1. The time complexity of Dijkstra algorithm is given by $\mathcal{O}(|\mathcal{E}| + |\mathcal{V}|\log|\mathcal{V}|)$. In the worst case, the algorithm needs to run $|\mathcal{V}^*|$ times, which is the number of destination satellites, and the total time complexity deteriorates to $\mathcal{O}(|\mathcal{V}^*|(|\mathcal{E}| + |\mathcal{V}|\log|\mathcal{V}|))$. This signifies that the complexity is contingent upon the structure of both the service request graph and the space network topology.

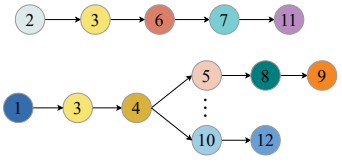
Complexity of Algorithm 2. The time complexity of merging these distinct sets into one set is $\mathcal{O}(\mathrm{n})$, where n is the total number of elements across all sets. Therefore, the time complexity of obtaining the minimum DST is inferred as $\mathcal{O}(|\mathcal{V}^*|(|\mathcal{E}| + |\mathcal{V}|\log|\mathcal{V}|) + 2^*\mathrm{n} + (|\mathcal{V}_z| + |\mathcal{E}_z|))$, where $\mathcal{O}(2^*\mathrm{n})$ represents the time complexity of merging the generated vertex and edge sets, and $\mathcal{O}(|\mathcal{V}_z| + |\mathcal{E}_z|)$ is the time complexity of constructing the directed Steiner tree.

VII. Performance Evaluation

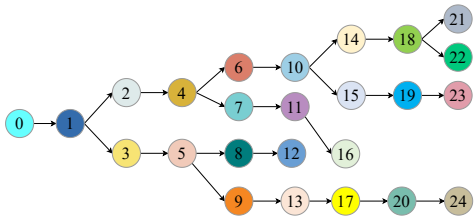
A. Experiment Setup

Constellation. In experiments, we consider a 72/12/1 Telesat Star Constellation system, which consists of $P = 6$ polar orbit planes whose inclination is 99.5°, and the orbital altitude is $h = 1000km$ on the ground [38].

Satellite Computing Capability. We suppose LEO satellites are equipped with several onboard processors and storage components. According to [27], we assign the satellite computing capability following a uniform distribution, as $\varphi(v_k) \sim \mathrm{DiscreteUniform}(\{100, 160, 220, 280, 340, 400\})$, while the processor power $\Psi(v_k) = \varphi(v_k) \times 0.01$ in Watts.



(a) Chained and Treed topologies.



(b) Sophisticated-Treed (Soph-Treed) topology.

Fig. 5: The overview of service request topologies from terrestrial stations.

ISL Parameters. Similar to settings in [32], [33], [39], the transmission power of all relevant satellites is standardized at $P_T = 3$ (Watt). In the context of the Telesat Constellation, which achieve inter-satellite interconnection across free-space optical link, the carrier frequency of each of satellite is set as f_c=193(THz), , and the corresponding carrier wavelength is λ=1550(nm) while the channel bandwidth between two satellites is defined as BW=2%f_c. Besides, we specify the antenna diameter transceiver as D_R=6(mm), the pointing loss angle $\theta_0 = 0.01\pi$ and the optical conversion efficiency of the transceiver modules $\eta = 0.8$.

Microservices. According to the microservice setting adopted by [27], we heuristically construct the microservices testbed as follows: There are $|\mathcal{M}| = 60$ microservices whose data volume follows a Gaussian distribution with a mean of 50 and variance of 4 in Gbits. For each microservice, there are $4 \sim 6$ replicas distributed on satellites, while each satellite can concurrently host at most $\lfloor|\mathcal{M}| \times 6/72\rfloor$ microservices.

Ablation Solutions: We design the following benchmarks to evaluate the performance of our proposed algorithm.

1) Random: It randomly selects a satellite for each microservice and uses the Dijkstra algorithm to get the shortest path when dealing with relay links.
2) Computation greedy (Comp-greedy): It chooses the satellite that offers the minimum computation energy cost for executing each microservice. The relay routing policy is the same as Random policy.
3) Communication greedy (Comm-greedy): It picks the satellite pair with the minimum communication energy cost, including the potential relay links.

We define three service request topologies: the Chained topology, the multi-forked Treed topology, and the Sophisticated Treed (Soph Tree) topology, as depicted in Fig. 5.

To verify the effectiveness of our algorithm, we conducted a series of experiments on a space topology comprising 72 satellites, each hosting 60 microservices with 4∼6 replicas. We generated 100 service request chains, varying in length from 10 to 20, to simulate a chained microservice topology. We also constructed 100 random treed topologies, containing 20 to 45, featuring a stochastic number of leaf nodes and varying branch configurations. For each sophisticated treed topology, microservices were randomly assigned to nodes, and this assignment process was independently repeated 100 times. For each topology, energy cost data was collected from 100 samples. The final results were derived by averaging the energy cost values across these samples. It ensures robust and reliable performance evaluation by mitigating variations caused by randomness in the topology generation and microservice allocation processes.

B. Experimental Results

1) Effectiveness: As illustrated in Fig. 6(a), our proposed Steiner consistently outperforms all baseline methods across various microservices topology scenarios, and its advantages become more pronounced in more complex topologies. This enhanced performance is charged to our comprehensive approach that simultaneously considers both computation and communication costs when determining the most efficient routing paths. The comp-greedy method outperforms the random method by prioritizing minimizing computation costs. The comm-greedy method surpasses both comp-greedy and random methods by focusing on minimizing communication costs. These findings emphasize the necessity of accounting for both computation and communication factors to design effective and energy-conscious routing strategies for the SCPN.

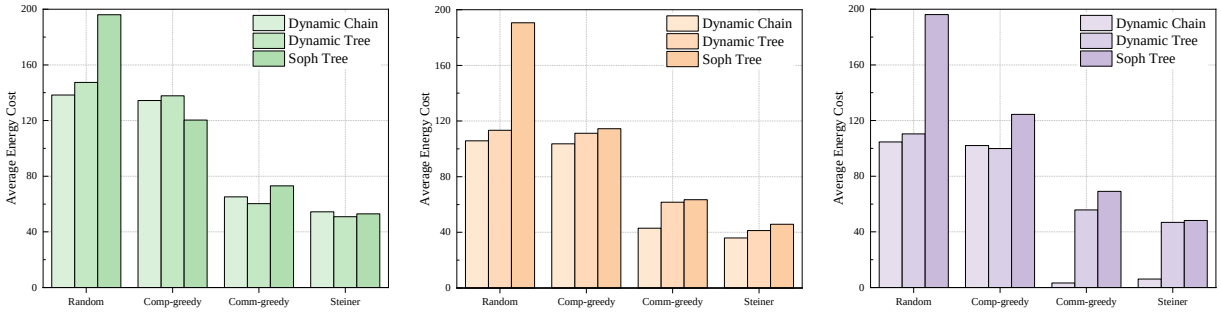
2) Impact of constellation scales: To evaluate the scalability of our algorithm, we systematically varied the number of satellites in the constellation by adjusting the number of orbits while maintaining consistent microservice deployment strategies. Experiments were conducted in two constellation settings separately, as illustrated in Fig. 6(b) and Fig. 6(c). These configurations allowed us to assess the algorithm's performance under varying levels of network complexity and service topology. Our results demonstrate that the proposed approach consistently outperforms alternative methods across all tested constellation scales and topological scenarios. Notably, its effectiveness becomes increasingly pronounced as the network complexity escalates, highlighting its robustness and adaptability to intricate environments.

3) Impact of transmitter power: Given the dominant role of communication costs in the total energy consumption of satellite systems, we conducted a series of experiments to explore how varying channel conditions influence the performance of our Steiner scheme. Specifically, we experimented with four different transmitter power levels on satellites across diverse microservice request schemes to examine their effect on energy efficiency. The results shown in Figs. 7(a), 7(b) and 7(c) indicate that as transmitter power increases, the average energy cost shows an upward trend. This trend highlights the trade-off between signal strength and energy consumption, with higher power levels leading to greater energy expenditure. Despite this increase in energy cost, our Steiner algorithm consistently outperformed other benchmark strategies across all service request scenarios. This superior performance is attributed to the algorithm's ability to dynamically adapt to the complexities of space network topologies and fluctuating channel conditions. By optimizing service routing paths, our approach minimizes unnecessary energy use under varying channel conditions. These findings underscore the importance of intelligent service routing mechanisms in the SCPN, particularly in environments with variable communication links.

4) Impact of faulty satellites: Given the constraints of limited power supply and the effects of solar eclipses, some satellites may need to enter a low-power mode, functioning solely as relay nodes for message forwarding [22]. To evaluate the robustness of our proposed method under such conditions, we conducted experiments by randomly disabling a subset of satellites in a constellation of 72 satellites. Specifically, we tested scenarios with 10, 20, and 25 faulty satellites by eliminating their microservices, as shown in Figs 8(a), 8(b), 8(c). The results reveal that energy cost generally increases with an increasing number of faulty satellites for all methods. This trend can be attributed to the reduced availability of active nodes, which forces longer transmission paths and higher energy consumption for maintaining connectivity (much more relay satellites are required for message forwarding). Despite this challenge, Steiner invariably outperforms other methods in all scenarios. Its superior performance is attributed to its robust design, which can effectively adapt to changes in network topology and operational conditions. By optimizing power allocation and service routing decisions, the algorithm minimizes disruptions caused by satellite failures.

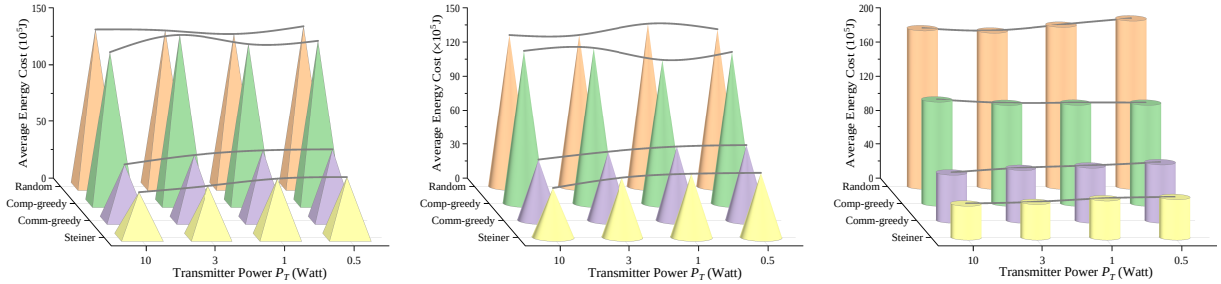
5) Impact of microservice deployment schemes: To investigate the influence of microservice deployment and scaling strategies on energy consumption with in the SCPN, we conducted experiments by varying the number of replicas allocated for each microservice. Specifically, we tested three replica ranges: $[2, 6]$, $[4, 6]$, and $[6, 10]$. And the experimental results, illustrated in Figs. 9(a), 9(b), and 9(c), reveal that energy costs generally decreases as the scale of microservice deployment expands. This trend is attributed to the increased availability of service nodes and routing paths, which provide more opportunities for route optimization and efficient power utilization. Our algorithm regularly demonstrates superior performance in reducing energy costs compared to other benchmarks, particularly in more complex microservice-satellite patterns. This superior performance can be attributed to the algorithm's ability to leverage the increased number of service routing paths and nodes, enabling more advanced route strategies. The expanded deployment range provides greater flexibility in service selections and routing decisions, allowing the algorithm to identify more energy-efficient configurations. By contrast, the energy costs associated with routing paths generated by random and comp-greedy methods occasionally increased as the number of replicas grew. This counterintuitive behavior can be attributed to inefficient routing decisions caused by the higher number of service nodes and calling paths, which complicate the decision-making process and lead to suboptimal resource utilization.

6) Adaptability and stability: Furthermore, as depicted in Fig 10, which presents the energy distribution derived from 300 independent experiments, underscores the superior stability of the Steiner policy compared to other



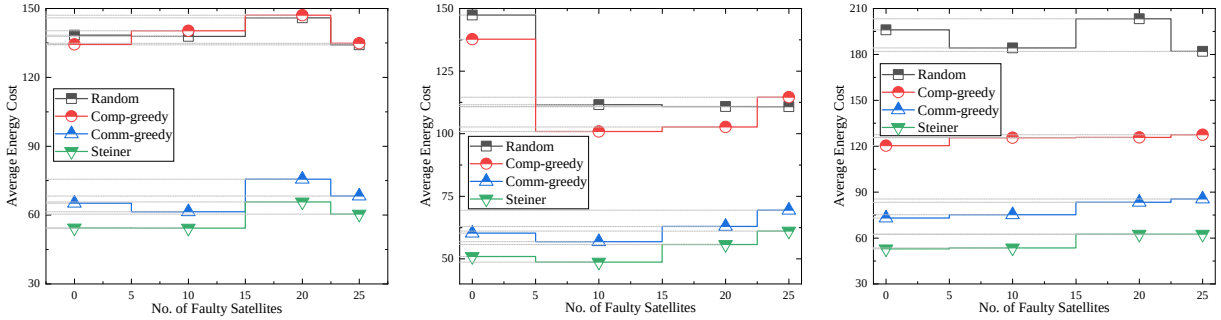
(a) Constellation with 72 satellites. (b) Constellation with 48 satellites. (c) Constellation with 108 satellites.

Fig. 6: Performance comparison on energy-cost under varying constellation scales for dynamic microservice request topologies.



(a) Chained service topology. (b) Treed service topology. (c) Sophisticated Treed service topology.

Fig. 7: Performance comparison on energy-cost under different onboard transmitter power configurations for dynamic microservice request topologies.



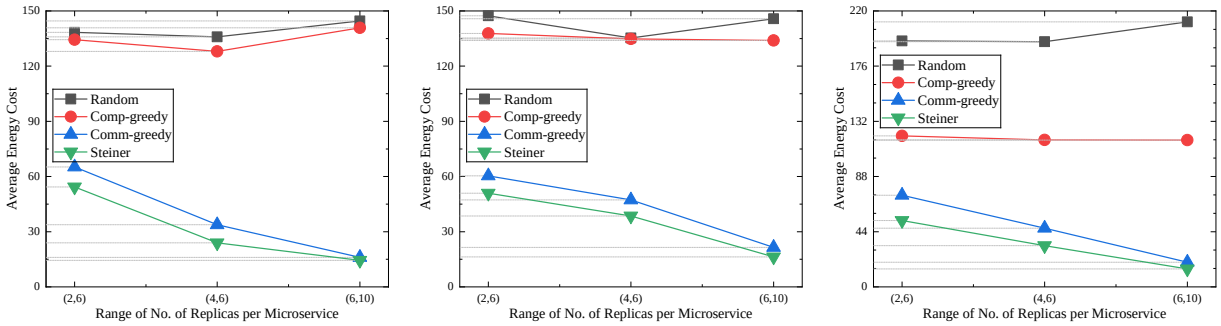
(a) Robustness over faulty satellites (Chain). (b) Robustness over faulty satellites (Tree). (c) Robustness over faulty satellites (Soph Tree).

Fig. 8: Performance comparison on energy-cost under different configurations on number of faulty satellites for dynamic service request topologies.

benchmark strategies. The results reveal that, even under sophisticated treed topologies with heightened complexity, the Steiner policy exhibits less variance in energy costs. This stability highlights its robustness and adaptability in managing diverse network conditions and service requirements.

VIII. Conclusion

In this paper, we proposed a DST-based solution to provide an energy-efficient service routing scheme for tasks from terrestrial clients. We systematically developed the SCPN models, including communication and computation, covering various aspects of the LEO satellite network. We also mathematically formulated the energy optimization problem for satellite-microservice routing and transformed it into finding a feasible DST solution within an augmented directed satellite-microservice colocated graph. To approximate the optimal solution, we designed a series of heuristic algorithms. Experimental results show that our algorithm significantly outperforms other benchmarks in energy efficiency, robustness, and adaptability across different network and microservice configurations in the context of SCPN.



(a) Microservice scale (Chain). (b) Microservice scale (Tree). (c) Microservice scale (Soph Tree).

Fig. 9: Performance comparison on energy-cost under different configurations on number of replicas of microservices for dynamic service request topologies.

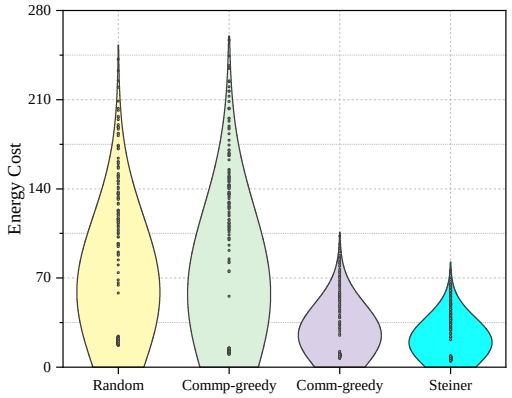


Fig. 10: Energy distribution of Soph Tree.

References

[1] Z. Xiao, J. Yang, T. Mao, C. Xu, R. Zhang, Z. Han, and X.-G. Xia, "Leo satellite access network (leo-san) towards 6g: Challenges and approaches," IEEE Wireless Communications, 2022.

[2] S. Ma, Y. C. Chou, H. Zhao, L. Chen, X. Ma, and J. Liu, "Network characteristics of leo satellite constellations: A starlink-based measurement from end users," in IEEE INFOCOM 2023 - IEEE Conference on Computer Communications, pp. 1–10, 2023.

[3] W. Mi and Y. Fang, "Intelligent remote sensing satellite and remote sensing image real-time service," Acta Geodaetica et Cartographica Sinica, vol. 48, no. 12, p. 1586, 2019.

[4] Z. Shen, J. Jin, C. Tan, A. Tagami, S. Wang, Q. Li, Q. Zheng, and J. Yuan, "A survey of next-generation computing technologies in space-air-ground integrated networks," ACM Comput. Surv., vol. 56, no. 1, pp. 1–40, 2023.

[5] H. Al-Hraishawi, H. Chougrani, S. Kisseleff, E. Lagunas, and S. Chatzinotas, "A survey on nongeostationary satellite systems: The communication perspective," IEEE Communications Surveys & Tutorials, vol. 25, no. 1, pp. 101–132, 2023.

[6] B. Zhang, Y. Wu, B. Zhao, J. Chanussot, D. Hong, J. Yao, and L. Gao, "Progress and challenges in intelligent remote sensing satellite systems," IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, vol. 15, pp. 1814–1822, 2022.

[7] H. Guo, J. Li, J. Liu, N. Tian, and N. Kato, "A survey on space-air-ground-sea integrated network security in 6g," IEEE Communications Surveys & Tutorials, vol. 24, no. 1, pp. 53–87, 2022.

[8] Z. Shen, J. Jin, C. Tan, A. Tagami, S. Wang, Q. Li, Q. Zheng, and J. Yuan, "A survey of next-generation computing technologies in space-air-ground integrated networks," ACM Comput. Surv., vol. 56, aug 2023.

[9] Y. Shi, L. Zeng, J. Zhu, Y. Zhou, C. Jiang, and K. B. Letaief, "Satellite federated edge learning: Architecture design and convergence analysis," IEEE Transactions on Wireless Communications, pp. 1–1, 2024.

[10] Y. Li, M. Wang, K. Hwang, Z. Li, and T. Ji, "Leo satellite constellation for global-scale remote sensing with on-orbit cloud AI computing," IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens., vol. 16, pp. 9369 – 9381, 2023.

[11] F. Tronchetti, "Legal aspects of satellite remote sensing," in Handbook of Space Law, pp. 501–553, Edward Elgar Publishing, 2015.

[12] M. Jia, L. Zhang, J. Wu, S. Meng, and Q. Guo, "Collaborative satellite-terrestrial edge computing network for everyone-centric customized services," IEEE Netw., vol. 37, no. 5, pp. 197–205, 2022.

[13] I. Leyva-Mayorga, M. Martinez-Gost, M. Moretti, A. Pérez-Neira, M. Á. Vázquez, P. Popovski, and B. Soret, "Satellite edge computing for real-time and very-high resolution earth observation," IEEE Transactions on Communications, vol. 71, no. 10, pp. 6180–6194, 2023.

[14] D. Merkel et al., "Docker: lightweight linux containers for consistent development and deployment," Linux j, vol. 239, no. 2, p. 2, 2014.

[15] T. Salah, M. J. Zemerly, C. Y. Yeun, M. Al-Qutayri, and Y. Al-Hammadi, "The evolution of distributed systems towards microservices architecture," in 2016 11th International Conference for Internet Technology and Secured Transactions (ICITST), pp. 318–325, IEEE, 2016.

[16] A. Ruiz-Zafra, J. Pigueiras-del Real, M. Noguera, L. Chung, D. G. Barres, and K. Benghazi, "Servitization of customized 3d assets and performance comparison of services and microservices implementations," IEEE Transactions on Services Computing, vol. 17, no. 1, pp. 194–208, 2024.

[17] Q. Ouyang, N. Ye, J. Gao, A. Wang, and L. Zhao, "Joint in-orbit computation and communication for minimizing download time from leo satellites," IEEE Transactions on Mobile Computing, vol. 23, no. 5, pp. 3950–3963, 2024.

[18] J. Wang, "Towards geoai as a containerized microservice," in Proceedings of the 31st ACM International Conference on Advances in Geographic Information Systems, pp. 1–2, 2023.

[19] N. Saeed, A. Elzanaty, H. Almorad, H. Dahrouj, T. Y. Al-Naffouri, and M.-S. Alouini, "Cubesat communications: Recent advances and future challenges," IEEE Communications Surveys & Tutorials, vol. 22, no. 3, pp. 1839–1862, 2020.

[20] H. Jia, Z. Ni, C. Jiang, L. Kuang, and J. Lu, "Uplink interference and performance analysis for megasatellite constellation," IEEE Internet of Things Journal, vol. 9, no. 6, pp. 4318–4329, 2022.

[21] C. Wang, Y. Zhang, Q. Li, A. Zhou, and S. Wang, "Satellite computing: A case study of cloud-native satellites," in 2023 IEEE International Conference on Edge Computing and Communications (EDGE), pp. 262–270, 2023.

[22] Y. Yang, M. Xu, D. Wang, and Y. Wang, "Towards energy-efficient routing in satellite networks," IEEE Journal on Selected Areas in Communications, vol. 34, no. 12, pp. 3869–3886, 2016.

[23] Y. Zhang, Q. Wu, Z. Lai, Y. Deng, H. Li, Y. Li, and J. Liu, "Energy drain attack in satellite internet constellations," in 2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS), pp. 1–10, 2023.
[24] K. Peng, L. Wang, J. He, C. Cai, and M. Hu, "Joint optimization of service deployment and request routing for microservices in mobile edge computing," IEEE Transactions on Services Computing, vol. 17, no. 3, pp. 1016–1028, 2024.
[25] Y. Yu, J. Yang, C. Guo, H. Zheng, and J. He, "Joint optimization of service request routing and instance placement in the microservice system," Journal of Network and Computer Applications, vol. 147, p. 102441, 2019.
[26] W. Guo, H. Hu, and Y. Liu, "Service function chain optimization deployment mechanism based on node and link influence," in 2019 IEEE 7th International Conference on Computer Science and Network Technology (ICCSNT), pp. 385–389, 2019.
[27] J. Cao, S. Zhang, Q. Chen, H. Wang, M. Wang, and N. Liu, "Computing-aware routing for leo satellite networks: A transmission and computation integration approach," IEEE Transactions on Vehicular Technology, vol. 72, no. 12, pp. 16607–16623, 2023.
[28] A. Fu, E. Modiano, and J. Tsitsiklis, "'optimal energy allocation and admission control for communications satellites," IEEE/ACM Transactions on Networking, vol. 11, no. 3, pp. 488–500, 2003.
[29] M. Charikar, C. Chekuri, T. yat Cheung, Z. Dai, A. Goel, S. Guha, and M. Li, "Approximation algorithms for directed steiner problems," Journal of Algorithms, vol. 33, no. 1, pp. 73–91, 1999.
[30] M. Medard, S. Finn, R. Barry, and R. Gallager, "Redundant trees for preplanned recovery in arbitrary vertex-redundant or edge-redundant graphs," IEEE/ACM Transactions on Networking, vol. 7, no. 5, pp. 641–652, 1999.
[31] N. Cheng, F. Lyu, W. Quan, C. Zhou, H. He, W. Shi, and X. Shen, "Space/aerial-assisted computing offloading for iot applications: A learning-based approach," IEEE Journal on Selected Areas in Communications, vol. 37, no. 5, pp. 1117–1129, 2019.
[32] I. Leyva-Mayorga, B. Soret, and P. Popovski, "Inter-plane inter-satellite connectivity in dense leo constellations," IEEE Transactions on Wireless Communications, vol. 20, no. 6, pp. 3430–3443, 2021.
[33] A. Polishuk and S. Arnon, "Optimization of a laser satellite communication system with an optical preamplifier," J. Opt. Soc. Am. A, vol. 21, pp. 1307–1315, Jul 2004.
[34] S. Ye, J. Du, L. Zeng, W. Ou, X. Chu, Y. Lu, and X. Chen, "Galaxy: A resource-efficient collaborative edge ai system for in-situ transformer inference," arXiv preprint arXiv:2405.17245, 2024.
[35] B. Shang, Y. Yi, and L. Liu, "Computing over space-air-ground integrated networks: Challenges and opportunities," IEEE Network, vol. 35, no. 4, pp. 302–309, 2021.
[36] Y. Gao, Z. Yan, K. Zhao, T. de Cola, and W. Li, "Joint optimization of server and service selection in satellite-terrestrial integrated edge computing networks," IEEE Transactions on Vehicular Technology, vol. 73, no. 2, pp. 2740–2754, 2024.
[37] T. Shake, J. Sun, T. Royster, and A. Narula-Tam, "Contingent routing using orbital geometry in proliferated low-earth-orbit satellite networks," in MILCOM 2022 - 2022 IEEE Military Communications Conference (MILCOM), pp. 404–411, 2022.
[38] I. Del Portillo, B. G. Cameron, and E. F. Crawley, "A technical comparison of three low earth orbit satellite constellation systems to provide global broadband," Acta astronautica, vol. 159, pp. 123–135, 2019.
[39] S. Nie and I. F. Akyildiz, "Channel modeling and analysis of inter-small-satellite links in terahertz band space networks," IEEE Transactions on Communications, vol. 69, no. 12, pp. 8585–8599, 2021.